Digital Signal Processing

TUTORIAL 09

CHIRP SIGNAL AND SHORT TIME FOURIER TRANSFORM

BIRMINGHAM CITY University

1. Learning Outcomes

- Understand the chirp signal algorithm.
- Be able to use FFT to analyse the signal.
- Be able to understand the Short-time Fourier transform.

The properties of Chirp signal

Chirp is the signal with linearly swept frequency. Most audio editing software has function to create or synthesis chirp signal. The following figure shows the chirp generation function of Audacity.

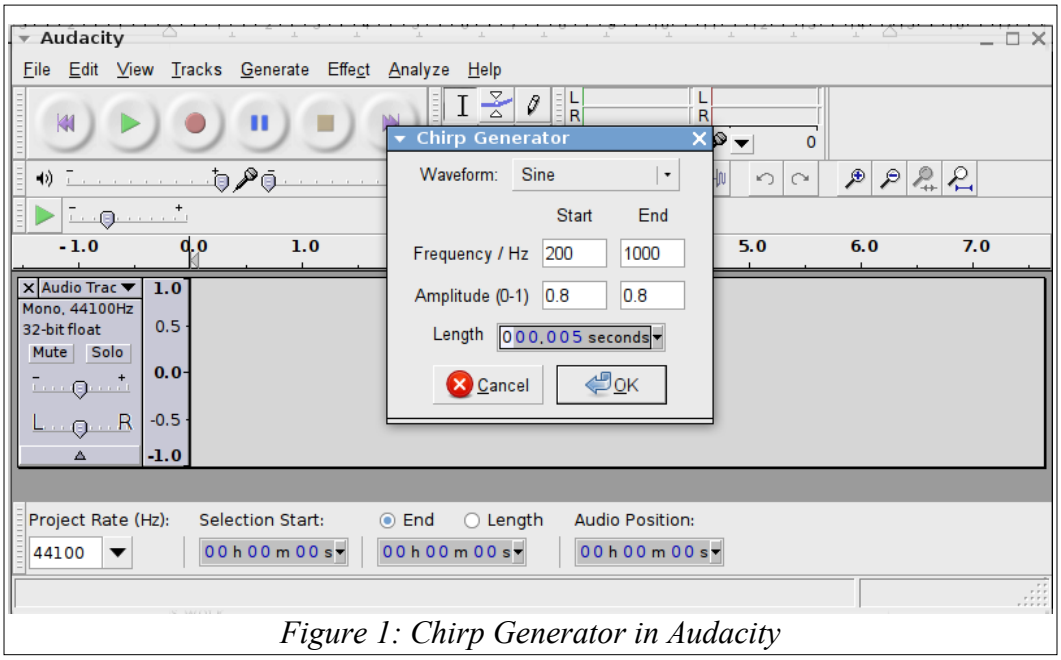

Figure 1: Chirp Generator in Audacity

Normally you can specify the start frequency, the end frequency, the amplitude and the length of chirp signal. For above example, we shall create 5 second long chirp beginning at 200 Hz and end at 1000 Hz.

Mathematically, the frequency "f" of chirp can be defined as the linear function of time "t". Since angular frequency "ω" is first derivative of angle "θ" against time. Therefore, if "θ" is angle which follows below equation (a, b, c are constants)

$\theta = 2\pi(at^2 + bt + c)$.

The angular frequency "ω" is going to be: $\omega = \theta' = 2\pi(at^2 + bt + c)'$which results in linear function of time "t". In this case:

$$\omega = \theta' = 2\pi(at^2 + bt + c)' = 2\pi(2at + b + 0)$$

$$f = \frac{\omega}{2\pi} = 2at + b$$

Therefore we got the relationship between the desired frequency "f" and coefficients "a" and "b".

Matlab code for creation of Chirp

Enter the following code into Matlab and you shall generate a wav file call 'mychirp1.wav' in same directory.

```
Fs=10000;
A=0.8;
Ts=1/Fs;
dur=1.5;
t=0:Ts:dur;
Theta=2*pi*(100+200*t+500*t.*t);
chirpsig=A*sin(Theta);
audiowrite('mychirp1.wav',chirpsig,Fs);
```

Table 1 Simple Chirp Signal Generation

This chirp begins at 200 Hz and end at 1700 Hz, use Matlab's signalAnalyzer spectrum tool to verify your results.

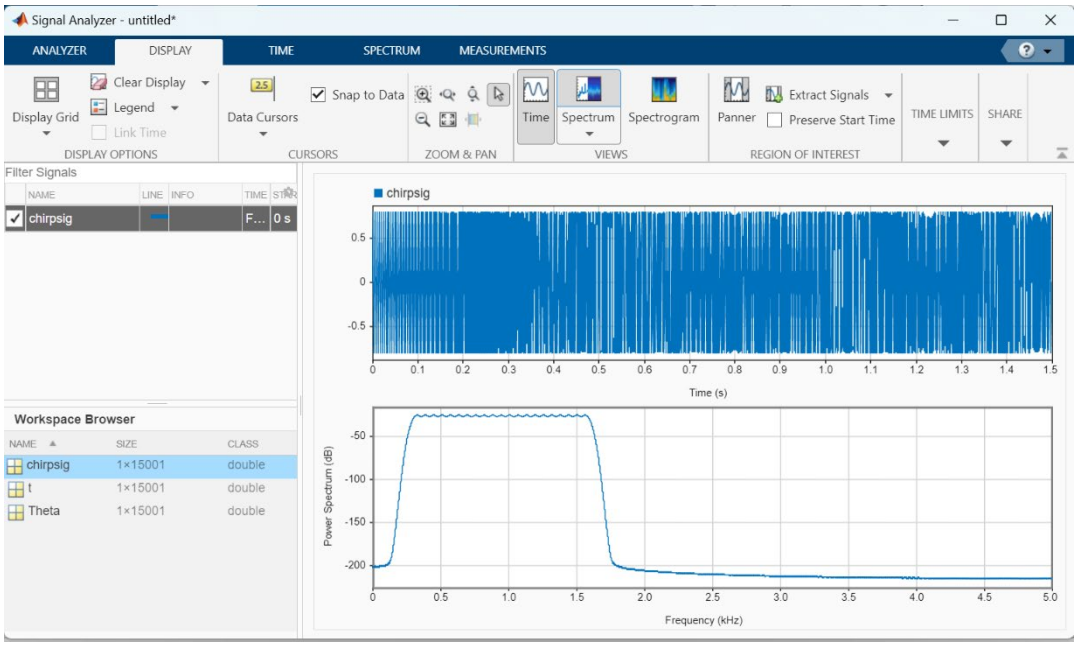

Figure 2 spectrum (FFT) of chirp signal

Add 'sound()' function at appropriate place to make the chirp sound. Please read the code and understand how start frequency and end frequency can be determined.

Here, the sixth line defined the angle "θ", which is a function of time "t". Comparing the equation "$\theta = at^2 + bt + c$", the coefficients are

$$a = 500; b = 200; c = 100$$

Note after deducting the first derivative of "θ", the coefficient $c = 100$ becomes zero. Therefore, the constant *'c'* does not affect the audible result.

FFT of reversed Chirp signal

The following code generates the chirp signal from 1700 Hz to 200 Hz, and plot the spectrum use FFT function. You can see the spectrum of this chirp signal is same as the one we generated on section 3.

```
%% 1700-200
Fs=10000;
A=0.8;
Ts=1/Fs;
dur=1.5;
t=0:Ts:dur;
Theta=2*pi*(100+1700*t-500*t.*t);
chirpsig=A*sin(Theta);
sound(chirpsig,Fs);
```

```
%% FFT of the second chirp signal
len = length(chirpsig);
SSC = fft(chirpsig);
SSR = abs(SSC)./len;
L=round(len/2);
Mag = mag2db(SSR);
f = (Fs/2)*(0:L)/L;
plot(f,Mag(1:L+1));
xlabel('Frequency')
ylabel('dB')
```

Table 2 FFT of Chirp Signal

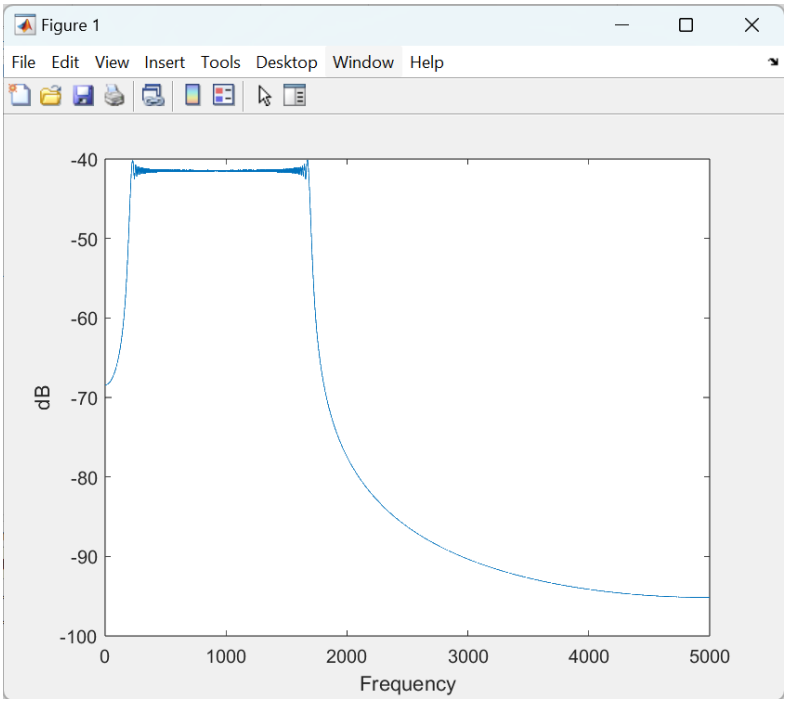

Figure 3 Manually plot FFT of Chirp

STFT

Since the frequency of the chirp signal various over the time, the normal spectrum estimation technique cannot represent the unique property of such signal. Therefore, the Short-Time Fourier Transform (STFT) is commonly used, which maps a signal into a two-dimensional function of time and frequency.

Within signalAnalyzer tool, you can plot the short time fourier transform of a signal with the spectrogram option. Try this this out with one of the chirp signals created previously.

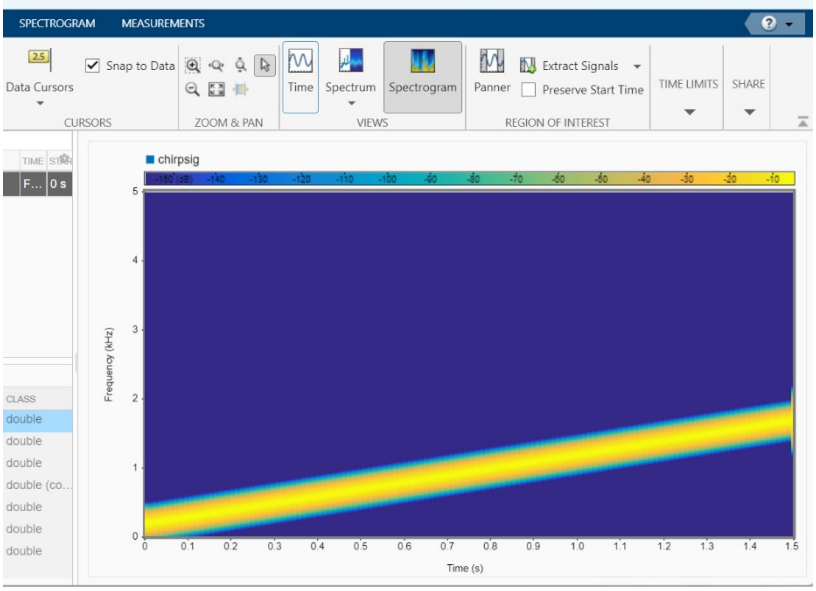

Figure 4: Spectrogram with signalAnalyzer

As we have seen previously, using a GUI is not always ideal, we can create STFTs using a command line or matlab script approach too.

The Matlab command “spectrogram” can provide a very simple way to generate STFT representation of the signal. The common syntax of “spectrogram” is

S = spectrogram(x,window,noverlap,nfft,fs)

For example the following code shall generate the spectrogram below in Figure :

```
spectrogram(chirpsig,128,120,128,10000);
```

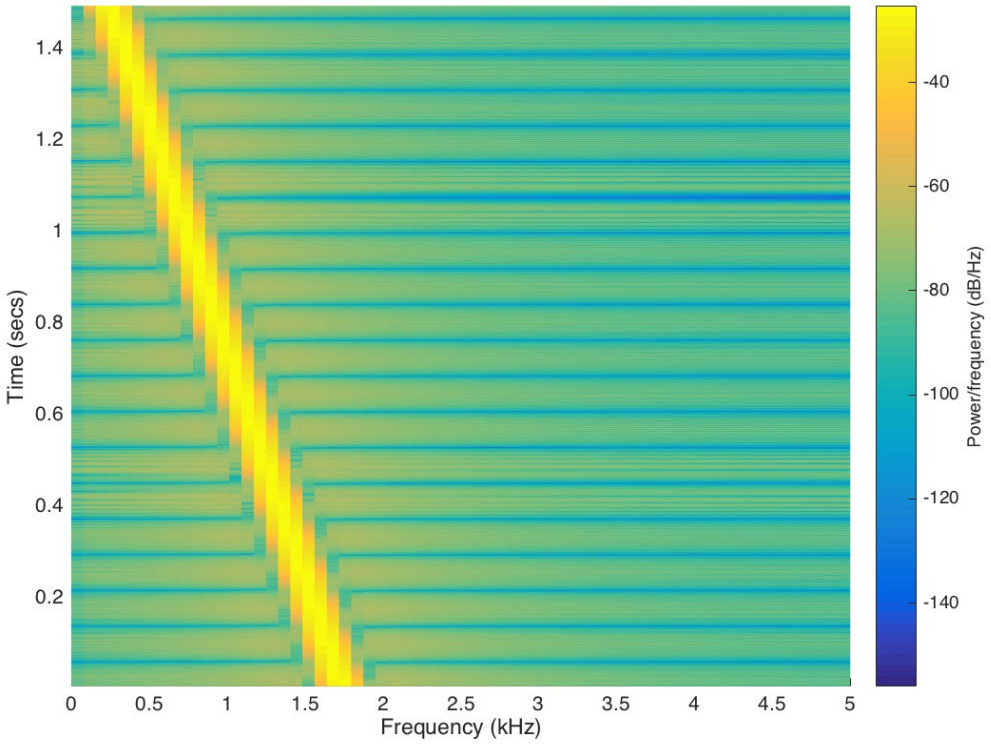

Figure 4 Spectrogram (STFT) of Chirp signal

Tasks

1) Function to create Chirp

To be able to simulate Audacity chirp generator function. Create a Matlab function which provides options to let a user input the desired start/end frequencies, and the duration of chirp. The following code implements this feature:

```
function signal = mychirp( f1, f2, dur, fs )
%MYCHIRP generate a linear-FM chirp signal
%
% usage: xx = mychirp( f1, f2, dur, fsamp )
%
% f1 = starting frequency
% f2 = ending frequency
% dur = total time duration
% fsamp = sampling frequency (OPTIONAL: default is 8000)
%
      if( nargin < 4 ) %-- Allow optional input argument
            fs = 8000;
      end
ts = 1/fs;
t = 0:ts:dur;
a = ??????????; % slope = 2u => u = slope/2
b = f1;
theta = 2*pi*(100 + b*t + a*t.*t);
signal = real(exp(j*theta));
end
```

Note: **exp(j*theta)** = $e^{j\theta} = cos(\theta) + isin(\theta)$

a. Complete above code and write some test code to test this function and output your result into wav file.

b. Read the code, understand it and explain the algorithm of this code.

2) Spectrum analysis using FFT code

Create your own function which uses matlab's FFT algorithm to generate and display the spectrum with correct magnitude and frequency units. Apply this to your chirp signal.

3) **Time Frequency Analysis using STFT** Investigate the Matlab help file of "*spectrogram",* try to use this command to generate the STFT diagram of your own chirp signals. Try to understand the parameters of "*window,noverlap,nfft"* and how they affect the result.